

# Intro to Machine Learning

*BA4 • 2025-2026*

# Contents

---

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>4</b>  |
| 1.1      | ML Basics . . . . .                                        | 4         |
| 1.1.1    | Rappels mathématiques . . . . .                            | 4         |
| <b>2</b> | <b>Linear Supervised ML</b>                                | <b>6</b>  |
| 2.1      | Linear regression . . . . .                                | 6         |
| 2.1.1    | Train . . . . .                                            | 6         |
| 2.1.2    | Test . . . . .                                             | 8         |
| 2.1.3    | Complement: multiple outputs . . . . .                     | 8         |
| 2.1.4    | Model evaluation on test sample . . . . .                  | 8         |
| 2.2      | Linear models for classification . . . . .                 | 9         |
| 2.2.1    | Logistic Regression . . . . .                              | 9         |
| 2.2.2    | Classification evaluation . . . . .                        | 11        |
| 2.2.3    | Dealing with multi class . . . . .                         | 13        |
| 2.2.4    | Evaluation metrics on multi-class classification . . . . . | 14        |
| 2.2.5    | SVM . . . . .                                              | 15        |
| <b>3</b> | <b>Non-linear Supervised ML</b>                            | <b>16</b> |
| 3.1      | k-Nearest Neighbors . . . . .                              | 16        |
| 3.1.1    | Distances . . . . .                                        | 16        |
| 3.1.2    | Classification . . . . .                                   | 16        |
| 3.1.3    | Regression . . . . .                                       | 17        |
| 3.2      | Feature expansion . . . . .                                | 17        |
| 3.2.1    | Polynomial curve fitting . . . . .                         | 17        |
| 3.2.2    | Fitting and regularization – Cross validation . . . . .    | 19        |
| 3.2.3    | Penalizing overfitting . . . . .                           | 20        |
| 3.3      | Kernel methods . . . . .                                   | 22        |
| 3.3.1    | Kernelizing . . . . .                                      | 23        |
| 3.3.2    | Linear regression . . . . .                                | 23        |
| 3.3.3    | Ridge regression . . . . .                                 | 24        |
| 3.3.4    | How to find a good $k(\cdot, \cdot)$ ? . . . .             | 24        |
| 3.3.5    | Multi-output form . . . . .                                | 24        |
| 3.4      | Artificial neural networks (deep learning) . . . . .       | 25        |

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>4</b> | <b>Unsupervised ML</b>             | <b>26</b> |
| 4.1      | Dimensionality reduction . . . . . | 26        |
| 4.2      | k-Means Clustering . . . . .       | 26        |

## CHAPTER 1

# Introduction

---

### 1.1 ML Basics

---

#### Definitions

- **Data sample:** an individual observation.
- **Data set:** a collection of multiple data samples.
- **Regression:** task on continuous values.
- **Classification:** task on discrete labels.

#### Notations

$$x_i \in \mathbb{R}^D$$

It is a column vector of dimension  $D$ .

$$y_i \in \mathbb{R}^C$$

It is a column vector of dimension  $C$ .

#### 1.1.1 Rappels mathématiques

##### Rappels

Si on a  $R(w)$  une fonction.

$$R'(\bar{w}) = \frac{dR}{dw}(\bar{w}) = \lim_{w \rightarrow 0} \frac{R(\bar{w} + w) - R(\bar{w})}{w}$$

Si

$$R'(\bar{w}) = 0$$

alors cela peut être :

- un minimum,
- un maximum,
- ou un point de selle.

On assumera dans la majorité des cas que la solution trouvée sera un minimum.

La solution à partir d'une telle équation, donc le  $w$  optimisé, sera dénoté

$$w^*$$

**Gradient**

$$\nabla_w R(w) = \begin{pmatrix} \frac{\partial R}{\partial w_1} \\ \frac{\partial R}{\partial w_2} \\ \vdots \\ \frac{\partial R}{\partial w_n} \end{pmatrix}$$

Si le gradient est égal au vecteur nul :

$$\nabla_w R(w) = 0$$

alors on obtient un *stationary point* lorsque l'on recherche  $w^*$ .

**Useful Identities**

$$\nabla_b(a^T b) = a$$

$$\|a\|^2 = a^T a$$

$$\nabla_a \|a\|^2 = 2a$$

$$\nabla_B(a^T B) = a$$

## CHAPTER 2

# Linear Supervised ML

---

## 2.1 Linear regression

---

Le principe de la régression linéaire est d'essayer de fitter une droite, un plan, etc. sur les données que nous avons afin de pouvoir en faire des approximations.

### 2.1.1 Train

On cherche à trouver le vecteur  $w$  optimal tel que :

$$\hat{y}_i = w^T x_i$$

Pour trouver ce  $w$ , on cherche le  $w$  optimal noté  $w^*$ , tel que les prédictions soient les plus proches des vraies valeurs.

#### Measure of closeness

Pour mesurer la distance entre la prédiction et la réalité, on utilise la distance euclidienne :

$$d(\hat{y}_i, y_i) = \sqrt{(\hat{y}_i - y_i)^2}$$

En pratique, on utilise la squared Euclidean distance :

$$d^2(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

Comme la racine carrée est monotone croissante sur les réels positifs, minimiser la distance ou son carré revient au même.

On va alors chercher donc à minimiser cette distance, pour être le plus proche possible des vrais résultats. \* Le training devient donc un minimization problem où on va tenter de minimiser ce que l'on définit alors la fonction de coût (empirical risk) :

$$R(w) = \frac{1}{N} \sum_{i=1}^N (w^T x_i - y_i)^2$$

Le problème de training devient donc :

$$\min_w R(w)$$

**Empirical Risk**

C'est ce que l'on va appeler ici la fonction de **EMPIRICAL RISK**  $R(w)$  dont la forme générale est :

$$R(\{x_i\}, \{y_i\}, w) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{y}_i, y_i)$$

Mais comment va-t-on réduire une telle somme facilement ? Grâce au gradient !!! On calcule ensuite le gradient en appliquant la dérivation terme à terme :

$$\nabla_w R(w) = \frac{1}{N} \sum_{i=1}^N \nabla_w ((w^T x_i - y_i)^2)$$

En utilisant la règle de dérivation (chaîne) :

$$\nabla_w (u^2) = 2u \nabla_w u$$

avec  $u = w^T x_i - y_i$ , et en notant que  $\nabla_w (w^T x_i) = x_i$ , on obtient :

$$\nabla_w R(w) = \frac{1}{N} \sum_{i=1}^N 2(w^T x_i - y_i) \nabla_w (w^T x_i)$$

Donc finalement :

$$\nabla_w R(w) = \frac{2}{N} \sum_{i=1}^N (w^T x_i - y_i) x_i$$

On impose la condition d'optimalité :

$$\nabla_w R(w) = 0$$

Ce qui donne :

$$\sum_{i=1}^N x_i x_i^T w^* = \sum_{i=1}^N x_i y_i$$

**Forme matricielle :**

$$X^T X w^* = X^T y$$

Si  $X^T X$  est inversible :

$$w^* = (X^T X)^{-1} X^T y$$

Sinon :

$$w^{\star} = X^{\dagger}y$$

### 2.1.2 Test

Une fois le modèle entraîné :

$$\hat{y} = w^{\star T}x$$

### 2.1.3 Complement: multiple outputs

On généralise à plusieurs sorties :

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

avec :

$$W = [w_1 \ w_2 \ \dots \ w_D]$$

La solution devient :

$$W^{\star} = X^{\dagger}Y$$

### Conclusion :

La régression linéaire permet d'obtenir une solution fermée exacte grâce à l'optimisation d'une fonction de coût quadratique convexe. Cela garantit que le minimum trouvé correspond à l'optimum global.

Ainsi, une fois  $W^{\star}$  calculé, la phase de test consiste simplement à appliquer :

$$\hat{y} = W^{\star T}x$$

Le modèle est donc entièrement déterminé par la solution analytique du problème d'optimisation.

### 2.1.4 Model evaluation on test sample

C'est bien de faire des modèles, mais il faut quand même vérifier s'ils sont bons. C'est pour cela qu'on utilise des métriques permettant d'évaluer les performances des modèles.



**Mean Squared Error (MSE)**

Le **Mean Squared Error** mesure la moyenne des erreurs au carré :

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

**Root Mean Squared Error (RMSE)**

Le **Root Mean Squared Error** est la racine du MSE :

$$\text{RMSE} = \frac{1}{N} \sum_{i=1}^N \sqrt{(y_i - \hat{y}_i)^2}$$

**Mean Absolute Error (MAE)**

Le **Mean Absolute Error** mesure la moyenne des erreurs absolues :

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i|$$

**Mean Absolute Percentage Error (MAPE)**

Le **Mean Absolute Percentage Error** mesure l'erreur relative en pourcentage :

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \left| \frac{\hat{y}_i - y_i}{y_i} \right|$$

## 2.2 Linear models for classification

### 2.2.1 Logistic Regression

Maintenant on veut un premier élément, une première méthode pour faire face aux problèmes de classification. C'est pour ça que l'on utilise la logistic regression.

#### *Linear classification*

On peut d'abord, avant de parler de logistic regression, utiliser la linear regression pour un problème de classification puis séparer les résultats en disant :

$$\hat{y} = \begin{cases} 1 & \text{if } y \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Mais cette méthode n'est pas fiable.

*LSF*

On va rester sur le même principe mais utiliser une fonction spéciale qui permet de séparer les différentes classes : la **sigmoid function**.

**Sigmoid function**

Définition de la sigmoid function :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Dans notre cas :

$$\hat{y}_i = \frac{1}{1 + \exp(-w^T x_i)} = \sigma(w^T x_i)$$

*Training*

Ici on doit trouver le  $w$  optimal noté  $w^*$ . Pour cela, on utilise les probabilités et on définit une loss function :

$$p(y|w) = \prod_{i=1}^N \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

En prenant le log, on obtient la empirical loss function :

$$R(w) = - \sum_{i=1}^N \left( y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i) \right)$$

On peut aussi écrire :

$$R(w) = - \sum_{i \in \text{positive}} \ln(\hat{y}_i) - \sum_{i \in \text{negative}} \ln(1 - \hat{y}_i)$$

Mais voilà ici, on ne sait pas exactement bien le résoudre facilement et pratiquement, on va donc utiliser un algorithme, le gradient descent. Mais d'abord calculer le gradient :

**Gradient sur les positive samples :**

$$\nabla_w \left( - \sum_{\text{positive}} \ln(\hat{y}_i) \right) = \sum_{\text{positive}} (\hat{y}_i - 1) x_i = \sum_{\text{positive}} (\hat{y}_i - y_i) x_i \quad (\text{ici } y_i = 1)$$

**Gradient sur les negative samples :**

$$\nabla_w \left( - \sum_{\text{negative}} \ln(1 - \hat{y}_i) \right) = \sum_{\text{negative}} \hat{y}_i x_i = \sum_{\text{negative}} (\hat{y}_i - y_i) x_i \quad (\text{ici } y_i = 0)$$

Ainsi, on peut les réunir et ça donne :

$$\nabla_w R(w) = \sum_{i=1}^N (\hat{y}_i - y_i) x_i$$

#### Gradient Descent Algorithm

1. Initialize  $w_0$  randomly
2. While not converged:

$$w_k \leftarrow w_{k-1} - \eta \frac{dR}{dw}(w_{k-1})$$

- $\eta$  est le learning rate
- on converge vers le minimum
- $\frac{dR}{dw}$  si on a multi-output sera remplacé par le gradient  $\nabla_w R(w_{k-1})$

#### Convergence criteria

- $|R(w_k) - R(w_{k-1})| < \delta_R$
- $\|w_k - w_{k-1}\| < \delta_w$
- ou nombre fixe d'itérations

#### Test

Une fois  $w^*$  trouvé :

$$\hat{y}_i = \sigma(w^{*T} x_i)$$

Puis classification :

$$\hat{y} = \begin{cases} 1 & \text{if } \hat{y}_i \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

#### 2.2.2 Classification evaluation

Ok c'est sympa de pouvoir faire de la classification maintenant mais on devrait quand même pouvoir évaluer comment on a fait si c'est bien ou pas. On utilise alors un tableau :

| Predicted class | True class |     |
|-----------------|------------|-----|
|                 | Pos        | Neg |
| Pos             | TP         | FP  |
| Neg             | FN         | TN  |

$$P = TP + FN \quad N = FP + TN$$

### Accuracy

**Definition:** % of correctly classified samples

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

### Precision

**Definition:** % of positives that are really positives on the said positives

$$\text{Precision} = \frac{TP}{TP + FP}$$

### Recall

**Definition:** % of positives correctly detected

$$\text{Recall} = \frac{TP}{TP + FN}$$

### False Positive Rate (FPR)

**Definition:** % of negatives incorrectly classified as positive

$$\text{FPR} = \frac{FP}{FP + TN}$$

**F1 Score**

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

**Interpretation:** The F1-score ranges between 0 and 1.

A value close to 1 indicates a very good balance between precision and recall, meaning few false positives and few false negatives.

A value close to 0 indicates a poor model performance, with either many false positives or many false negatives.

In practice:

- $F1 \approx 1$  : excellent classification performance
- $F1 \approx 0.5$  : medium performance, imbalance between precision and recall
- $F1 \approx 0$  : very poor model

**2.2.3 Dealing with multi class**

Maintenant, ce qu'on a fait pour la classification marchait seulement si on devait choisir entre 2 classes, par exemple 0 et 1. Mais comment fait-on lorsque l'on a plusieurs classes ?

On représente alors les données différemment :  $y_i$  devient un vecteur multidimensionnel contenant des 0 et un unique 1 à la position correspondant au label.

**This is known as the ONE-HOT ENCODING.**

Exemple :

$$y_i = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

In this case, we are of the label 3.

*Linear classification*

La méthode de train devient :

$$\min_W \sum_{i=1}^N \|W^T x_i - y_i\|^2$$

dont la solution est :

$$W^* = X^\dagger Y$$

Puis :

$$\hat{y}_i = W^{\star T} x_i$$

Dans ce cas, le vecteur  $\hat{y}_i$  donne un score pour chaque catégorie.

On choisit alors la catégorie ayant le plus grand score :

$$k = \arg \max_j \hat{y}^{(j)}$$

### *Logistic regression*

On utilise maintenant la fonction **Softmax** :

$$\hat{y}^{(k)}(x) = \frac{\exp(w^{(k)T}x)}{\sum_{j=1}^C \exp(w^{(j)T}x)}$$

Avec :

- $w^{(k)}$  : la  $k$ -ème colonne de  $W$
- $C$  : le nombre de classes

L'empirical risk devient :

$$R(W) = - \sum_{i=1}^N \sum_{k=1}^C y_i^{(k)} \ln(\hat{y}_i^{(k)})$$

Et on applique ensuite le gradient descent algorithm avec :

$$\nabla_W R(W) = \sum_{i=1}^N x_i (\hat{y}_i - y_i)^T$$

Le résultat final est alors obtenu avec :

$$k = \arg \max_j \hat{y}^{(j)}$$

#### 2.2.4 Evaluation metrics on multi-class classification

Etant donnée qu'on a plusieurs classes, les éléments qu'on avait définis (*Accuracy*, *Recall*, *Precision*...) pour évaluer les modèles de classification ne marchent plus vraiment.

Pour ce faire, on change déjà le tableau pour construire la **CONFUSION MATRIX**.

**Example with 3 labels**

|          | <i>A</i> | <i>B</i> | <i>C</i> |
|----------|----------|----------|----------|
| <i>A</i> | ...      | ...      | ...      |
| <i>B</i> | ...      | ...      | ...      |
| <i>C</i> | ...      | ...      | ...      |

Predicted c'est le ABC en ligne

Actual c'est le ABC en colonne (à gauche)

Par exemple, pour l'Accuracy, la formule devient :

$$\text{Accuracy} = \frac{\sum \text{true good guesses}}{\text{total}}$$

Pour Precision, Recall, FPrate et F1-score, on peut :

- do for each label and then the average

But that solution does not account for **class imbalance**.

### 2.2.5 SVM

ça j'ai pas bien compris du coup je préfère te laisser remplir

## CHAPTER 3

# Non-linear Supervised ML

---

### 3.1 k-Nearest Neighbors

KNN va chercher à déterminer la valeur en fonction des éléments à côté, des neighbors, pour pouvoir déterminer la valeur dudit point.

C'est très sommaire mais ça marche donc c'est sympa !

#### 3.1.1 Distances

Pour déterminer les éléments à côté il faut savoir à quelle distance on est d'eux, donc on doit choisir des mesures de distance.

##### Euclidian distance

**Definition:** mesure de la distance géométrique entre deux points.

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^D (x_i^{(k)} - x_j^{(k)})^2}$$

##### Chi-square distance

**Definition:** distance souvent utilisée pour comparer des distributions ou histogrammes.

$$d(x_i, x_j) = \sqrt{\chi^2(x_i, x_j)} = \sqrt{\sum_{k=1}^D \frac{(x_i^{(k)} - x_j^{(k)})^2}{x_i^{(k)} + x_j^{(k)}}}$$

#### 3.1.2 Classification

Pour la classification, c'est plutôt simple : on va prendre les  $k$  neighbors les plus proches puis choisir le label le plus fréquent.

##### KNN classification algorithm

1. Compute distance between  $x$  and all  $x_i$
2. Find  $k$  samples with minimum distances
3. Find the most common label among these  $k$  nearest neighbors
4. Assign the result



### 3.1.3 Regression

Pour la regression, on peut faire la moyenne des neighbors pour obtenir le résultat.

Pour faire ladite moyenne, on peut utiliser différentes méthodes : weighted ou non.

#### KNN regression algorithm

1. Compute distance between  $x$  and all  $x_i$
2. Find  $k$  samples with minimum distances
3. Compute the value based on the test sample using the K-NN:
  - average
  - weighted average
    - probability weighting
    - inverse distance weighting

$$\hat{y} = \frac{\sum_i w_i y_i}{\sum_i w_i}$$

avec :

$$w_i = \frac{1}{d_i}$$

## 3.2 Feature expansion

### 3.2.1 Polynomial curve fitting

On arrive à un nouveau défi.

Quand on représente les points, cela peut faire des courbes, des cercles, des exponentielles ou encore des polynômes très complexes.

Dans ces cas-là, une simple droite ou les KNN ne sont pas toujours très précis.

Pour cela, on utilise la **POLYNOMIAL CURVE FITTING** par **POLYNOMIAL CURVE EXPANSION**.

#### Base

En quoi cela consiste ?

On prend nos données de base  $x$  puis on leur applique une transformation  $\phi$  qui crée un nouveau vecteur contenant différentes combinaisons des paramètres initiaux.

Par exemple :

$$x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

Après polynomial curve expansion :

$$\phi(x) = \begin{pmatrix} x_1 \\ x_2 \\ x_1 + x_2 \\ x_1^2 \end{pmatrix}$$

C'est ça la polynomial curve expansion.

Grâce à cette transformation, on peut approximer des courbes beaucoup plus complexes même avec des méthodes similaires à la régression linéaire.

On obtient alors :

$$\hat{y}_i = w^T \phi(x_i)$$

Avec :

- $\phi(x_i) \in \mathbb{R}^F$
- $w \in \mathbb{R}^F$

### *Application*

On obtient alors :

$$\nabla_w R(w) = 2 \sum_{i=1}^N \phi(x_i) \left( \phi(x_i)^T w - y_i \right)$$

En imposant :

$$\nabla_w R(w) = 0$$

on obtient :

$$\sum_{i=1}^N \phi(x_i) \phi(x_i)^T w^* = \sum_{i=1}^N \phi(x_i) y_i$$

Donc :

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

**Cover's theorem**

*A complex pattern classification problem cast in a high-dimensional space non-linearly is more likely to be linearly separable than in a low-dimensional space, provided that the space is not densely populated.*

**3.2.2 Fitting and regularization – Cross validation****Definition: Overfitting**

Error low on training data but large on test data.

**Definition: Underfitting**

Error large on both training and testing data.

Jusqu'à maintenant on a vu plusieurs modèles et plusieurs manières de les comprendre, de les analyser et de voir les résultats.

Mais on n'a pas encore de méthode permettant de savoir si notre modèle est performant avant de le tester dans la vraie vie.

Il faut donc remédier à cela.

Pour cela, on va simuler training et test uniquement sur les training data puisque l'on n'est pas censé connaître les test data.

**Validation set**

Principe simple :

We separate the data into training and testing data, run the algorithm normally and then analyze the results.

We usually use ratios such as:

80/20      or      70/30

**Leave-one-out cross validation (LOOCV)**

For  $i = 1 \dots N$

1.  $(x_i, y_i)$  be the  $i$ -th sample
2. Temporarily remove  $(x_i, y_i)$  from the training set
3. Perform regression on the remaining  $N - 1$  samples
4. Compute the error on  $(x_i, y_i)$

When all points have been processed, report the average error.

**K-fold cross validation**

We split the data into  $k$  partitions.

Then, for each partition:

- train on the other partitions
- test on the selected partition
- compute the error

Finally, report the average error over all points.

*Pros and cons**Comparison*

- **Validation set:** cheap but may waste data
- **LOOCV:** expensive but does not waste data
- **K-fold:** good compromise between both methods

**3.2.3 Penalizing overfitting**

On va ajouter des regularisateurs pour avoir moins de chances d'obtenir des problèmes dans nos simulations, nos trainings et nos predictions.

On ajoute un regularizer à l'empirical risk.

Nouvelle training objective function :

$$E(w) = R(w) + \lambda E_w(w)$$

Avec :

- $\lambda > 0$  : hyperparameter controlling the influence of the regularizer
- $E_w(w)$  : regularizer

*Common regularizer***L2 regularization**

$$E_w(w) = w^T w$$

This regularizer leads to the regularized linear regression:

$$\min_w \sum_{i=1}^N \left( \phi(x_i)^T w - y_i \right)^2 + \lambda w^T w$$

On a :

$$\nabla_w E(w) = \nabla_w R(w) + \nabla_w E_w(w)$$

Ce qui donne :

$$\nabla_w E(w) = 2 \sum_{i=1}^N \phi(x_i) \left( \phi(x_i)^T w - y_i \right) + 2\lambda w$$

Donc la solution devient :

$$w^* = \left( \Phi^T \Phi + \lambda I_F \right)^{-1} \Phi^T y$$

**This is a RIDGE REGRESSION**

$I_F$  is the identity matrix of dimension  $F$ .

*Multi-output ridge regression*

On multi-output problems, it becomes:

$$\min_W \sum_{i=1}^N \|W^T \phi(x_i) - y_i\|^2 + \lambda \|W\|_F^2$$

Avec :

$$\|W\|_F$$

the Frobenius norm.

Consequently:

$$W^* = \left( \Phi^T \Phi + \lambda I_F \right)^{-1} \Phi^T Y$$

*What about SVM*

Pour les SVM, pas besoin d'ajouter explicitement de régularisation.

La régularisation est déjà préincluse dans la formulation initiale du SVM.

**3.3 Kernel methods**

Jusque-là on a pu voir comment améliorer nos algos, comment les différencier, les tester, et déterminer le meilleur.

Pendant cela, on a vu comment faire de la polynomial curve expansion et définir  $\phi$ .

Mais le problème est que cela reste complexe, et assez arbitraire lorsqu'il faut définir  $\phi$  et tester de nombreuses combinaisons.

C'est pourquoi on introduit un nouvel outil : **the KERNEL FUNCTION**.

Instead of explicitly defining the mapping  $\phi$ , we define a similarity function  $k(x_i, x_j)$ , defining a relation between points.

$k$  is a scalar (not a vector).

$$k(x_i, x_j) = \phi(x_i)^T \phi(x_j) \quad (\text{even if } \phi \text{ may be unknown})$$

Exemple :

$$k(x_i, x_j) = (x_i^T x_j + c)^d$$

is equivalent to a feature map:

$$\phi(x) = \begin{pmatrix} (x^{(1)})^2 \\ (x^{(2)})^2 \\ \sqrt{2} x^{(1)} x^{(2)} \\ \sqrt{2} x^{(1)} \\ \sqrt{2} x^{(2)} \\ 1 \end{pmatrix}$$

Exemple : Gaussian kernel

$$k(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

This kernel does not have a simple explicit  $\phi$  representation.

### 3.3.1 Kernelizing

Since a kernel function corresponds to a similarity (dot product), we can replace any dot product between samples in ML algorithms with a kernel function.

This is called **kernelizing an algorithm**.

$$k(X, x_i) = \begin{pmatrix} k(x_1, x_i) \\ k(x_2, x_i) \\ \vdots \\ k(x_N, x_i) \end{pmatrix} \in \mathbb{R}^N$$

Kernel matrix:

$$K = \begin{pmatrix} k(x_1, x_1) & \cdots & k(x_1, x_N) \\ \vdots & \ddots & \vdots \\ k(x_N, x_1) & \cdots & k(x_N, x_N) \end{pmatrix}$$

### 3.3.2 Linear regression

Normally:

$$w^* = (\Phi^T \Phi)^{-1} \Phi^T y$$

With kernels:

$$R(\alpha) = \sum_{i=1}^N \left( \alpha^T k(X, x_i) - y_i \right)^2$$

avec  $k(X, x_i) = \Phi \phi(x_i)$ .

Gradient:

$$\begin{aligned} \nabla_{\alpha} R(\alpha) &= 2 \sum_{i=1}^N k(X, x_i) \left( k(X, x_i)^T \alpha - y_i \right) \\ &= 2KK\alpha - 2Ky \end{aligned}$$

Solve:

$$K\alpha^* = y \quad \Rightarrow \quad \alpha^* = K^{-1}y$$

$$w^* = \Phi^T \alpha^*$$

Final prediction:

$$\hat{y} = w^{*T} \phi(x) = y^T K^{-1} k(X, x)$$

### 3.3.3 Ridge regression

$$E(w) = \sum (w^T \phi(x_i) - y_i)^2 + \lambda w^T w$$

Kernel form:

$$\begin{aligned} E(\alpha) &= \sum (\alpha^T \Phi \phi(x_i) - y_i)^2 + \lambda \alpha^T \Phi \Phi^T \alpha \\ &= \sum (\alpha^T k(X, x_i) - y_i)^2 + \lambda \alpha^T K \alpha \end{aligned}$$

Gradient:

$$\nabla_{\alpha} E(\alpha) = 2K(K + \lambda I_N)\alpha - 2Ky$$

Solution:

$$\alpha^* = (K + \lambda I_N)^{-1} y$$

### 3.3.4 How to find a good $k(\cdot, \cdot)$ ?

By cross validation (no miracle solution exists).

### 3.3.5 Multi-output form

$$E(W) = \sum \|W^T \phi(x_i) - y_i\|^2 + \lambda \|W\|_F^2$$

Solution:

$$W^* = \Phi^T (K + \lambda I)^{-1} Y$$

Prediction:



$$\hat{y} = W^*{}^T \phi(x) = Y^T (K + \lambda I)^{-1} k(X, x)$$

### 3.4 Artificial neural networks (deep learning)

*Section à compléter*

Cette section sera complétée ultérieurement.

## CHAPTER 4

# Unsupervised ML

---

### 4.1 Dimensionality reduction

---

*Section à compléter*

Cette section sera complétée ultérieurement.

### 4.2 k-Means Clustering

---

*Section à compléter*

Cette section sera complétée ultérieurement.